successfully created as follows. The value is generated randomly by the script:

```
$ kubectl get secrets polar-keycloak-client-credentials
```

NOTE The Keycloak Helm chart spins up a PostgreSQL instance inside the cluster and uses it to persist the data used by Keycloak. We could have integrated it with the PostgreSQL service managed by DigitalOcean, but the configuration on the Keycloak side would have been quite complicated. If you'd like to use an external PostgreSQL database, you can refer to the Keycloak Helm chart documentation (https://bitnami.com/stack/keycloak/helm).

The Keycloak server is deployed in a dedicated `keycloak-system` namespace. Applications can interact with it at `polar-keycloak.keycloak-system.svc.cluster.local` on port `8080` from within the cluster. It's also exposed outside the cluster via a public IP address. You can find the external IP address with the following command:

```
$ kubectl get service polar-keycloak -n keycloak-system

NAME             TYPE           CLUSTER-IP       EXTERNAL-IP
polar-keycloak   LoadBalancer   10.245.191.181   <external-ip>
```

The platform might take a few minutes to provision a load balancer. During the provisioning, the `EXTERNAL-IP` column will show a `<pending>` status. Wait and try again until an IP address is shown. Note it down, since we're going to use it in multiple scenarios.

Since Keycloak is exposed via a public load balancer, you can use the external IP address to access the Admin Console. Open a browser window, navigate to http://<external-ip>/admin, and log in with the credentials returned by the previous deployment script.